컴파일러의 기초 project 2 보고서

전기정보공학부 2021-15905 홍성준

October 16, 2025

1 Baseline

먼저, 문서에 정리된 operator precedence 및 associativity rule을 따라 아래 Figure 2와 같이 `subc.y`에 선언하였다. 여러 precedence를 가질 수 있는 terminal인 `++`, `--`, `-`, `*`은 일단 낮은 precedence로 선언한 후 grammar rule section에서 precedence를 높이는 방식으로 구현하였다.

```
/* Precedences and Associativities */
%left ','
%right '='
%left LOGICAL_OR
%left LOGICAL_AND
%left EQUOP
%left RELOP
%left '+' '-'
%left '*' '/' '%'
%right INCOP DECOP '!' '&'
%left '(' ')' '.' STRUCTOP '[' ']'
```

Figure 1: Baseline of precedence & associativity rule

2 Dangling-Else Problem

두 개의 production

(1) `stmt -> IF '(' expr ')' stmt ELSE stmt`

(2) `stmt -> IF '(' expr ')' stmt`

에 대해 shift/reduce conflict가 발생한다. 이는 `stmt`까지 shift한 이후, production (2)를 사용해 `stmt`로 reduce할지, production (1)을 위해 ELSE를 shift 해야 할지 정해주지 않았기 때문이다. 따라서 아래 Figure 2와 같이 `ELSE`의 precedence를 가장 높게 설정해주었고, Figure 3처럼 `ELSE`보다 precedence가 낮은 `'('`를 이용해 production (2)의 precedence를 ELSE보다 낮게 만들어 주었다. 최종적으로,

dangling-else의 conflict에선 else가 가까운 if와 match되도록 ELSE로 shift하게 함으로써 shift/reduce conflict를 해결할 수 있었다.

```
/* Precedences and Associativities */
%left ','
%right '='
%left LOGICAL_OR
%left LOGICAL_AND
%left EQUOP
%left RELOP
%left '+' '-'
%left '*' '/' '%'
%right INCOP DECOP '!' '&'
%left '(' ')' '.' STRUCTOP '[' ']'

/* precedence rule for dangling else problem */
%precedence ELSE
```

Figure 2: Declaration of ELSE precedence

```
stmt
  : expr ';'                                        { REDUCE("stmt->expr \';\'"); }
  | compound_stmt                                   { REDUCE("stmt->compound_stmt"); }
  | RETURN ';'                                      { REDUCE("stmt->RETURN \';\'"); }
  | RETURN expr ';'                                 { REDUCE("stmt->RETURN expr \';\'"); }
  | ';'                                             { REDUCE("stmt->\';\'"); }
  | IF '(' expr ')' stmt %prec '('                  { REDUCE("stmt->IF \'(\' expr \')\' stmt"); }
  | IF '(' expr ')' stmt ELSE stmt                  { REDUCE("stmt->IF \'(\' expr \')\' stmt ELSE stmt"); }
  | WHILE '(' expr ')' stmt                         { REDUCE("stmt->WHILE \'(\' expr \')\' stmt"); }
  | FOR '(' expr_e ';' expr_e ';' expr_e ')' stmt   { REDUCE("stmt->FOR \'(\' expr_e \';\' expr_e \';\' expr_e \')\' stmt"); }
  | BREAK ';'                                       { REDUCE("stmt->BREAK \';\'"); }
  | CONTINUE ';'                                    { REDUCE("stmt->CONTINUE \';\'"); }
  ;
```

Figure 3: Adjusted `stmt` production

3 INCOP unary / unary '[' Conflict

두 개의 production

(1) `unary -> INCOP unary`

(2) `unary -> unary '[' expr ']'`

에 대해 shift/reduce conflict가 발생한다. 이는 `INCOP`가 아닌 `DECOP`일 경우에도 동일하게 conflict가 발생한다. 이런 conflict가 발생하는 이유는 `INCOP unary` 이후 `[`을 만났을 때, `[`를 shift할지 아니면 production (1)에 따라 `unary`로 reduce할지 정해지지 않았기 때문이다. 이를 해결하기 위해 Figure 2와 같이 `[`, `]` terminal의 precedence를 `INCOP` 및 `DECOP`보다 더 크게 설정하였고, production (2)를 위해 shift를 선택하도록 함으로써 shift/reduce conflict를 해결할 수 있었다.